LAB SESSION 2

Programming Exercise:

Question 1: Any character is entered by the user; write a program to determine whether the character entered is a capital letter, a small case letter, a digit or a special symbol.

C: > C++ PROGRAMS > tut1.cpp > ...

```cpp
int main()
    if (ch >= 'A' && ch <= 'Z')
    else if (ch >= 'a' && ch <= 'z')
    {
        cout << "It is a small case letter." << endl;
    }
    else if (ch >= '0' && ch <= '9')
    {
        cout << "It is a digit." << endl;
    }
    else
    {
        cout << "It is a special symbol." << endl;
    }

    return 0;
}
```

```
PS C:\Users\hp> cd "c:\C++ PROGRAMS\"
 ; if ($?) { g++ tut1.cpp -o tut1 } ;
 if ($?) { .\tut1 }
Enter any character: a
It is a small case letter.
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRA
MS\" ; if ($?) { g++ tut1.cpp -o tut1
 } ; if ($?) { .\tut1 }
Enter any character: A
It is a capital letter.
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRA
MS\" ; if ($?) { g++ tut1.cpp -o tut1
 } ; if ($?) { .\tut1 }
Enter any character: 1
It is a digit.
PS C:\C++ PROGRAMS>
```

Question 2: Write a C++ program to generate the multiplication table of a given number.

C: > C++ PROGRAMS > tut1.cpp > main()

```cpp
#include <iostream>
using namespace std;

int main()
{
    int num;

    cout << "Enter a number to generate its multiplication table: ";
    cin >> num;

    cout << "Multiplication table of " << num << ":\n";

    for (int i = 1; i <= 10; i++)
    {
        cout << num << " x " << i << " = " << num * i << endl;
    }

    return 0;
}
```

```
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRA
MS\" ; if ($?) { g++ tut1.cpp -o tut1
 } ; if ($?) { .\tut1 }
Enter a number to generate its multip
lication table: 9
Multiplication table of 9:
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
9 x 10 = 90
PS C:\C++ PROGRAMS>
```

Question 3: Write a program to calculate the monthly telephone bills as per the following rule:

Minimum Rs. 200 for upto 100 calls.

Plus Rs. 0.60 per call for next 50 calls.

Plus Rs. 0.50 per call for next 50 calls.

Plus Rs. 0.40 per call for any call beyond 200 calls.

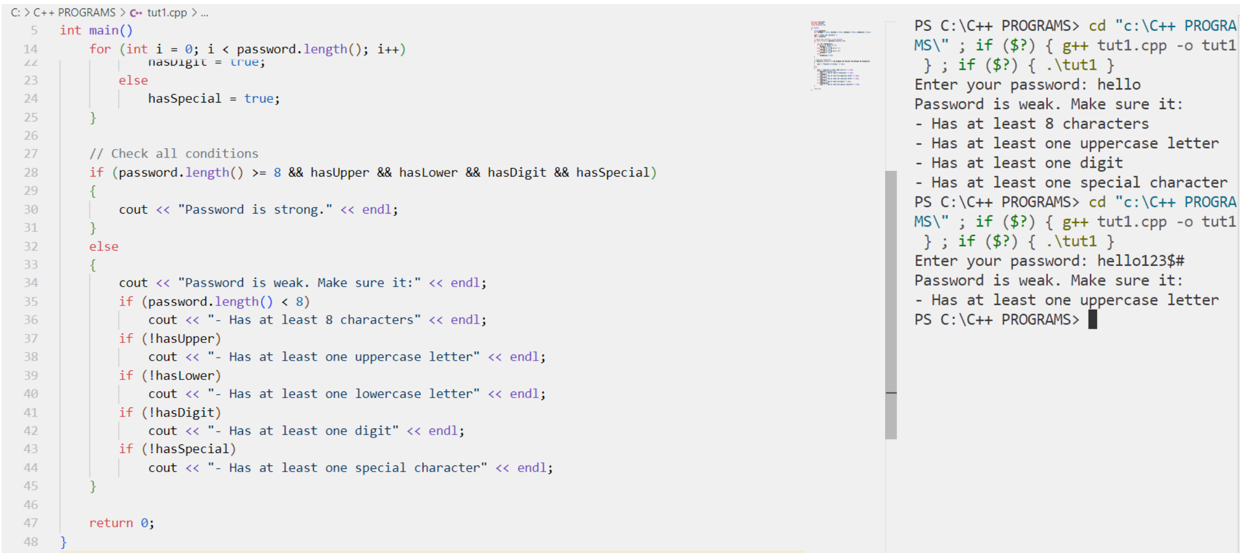
C: > C++ PROGRAMS > tut1.cpp > ...

```cpp
#include <iostream>
using namespace std;

int main()
{
    int calls;
    float bill;

    cout << "Enter the number of calls made: ";
    cin >> calls;

    if (calls <= 100)
    {
        bill = 200;
    }
    else if (calls <= 150)
    {
        bill = 200 + (calls - 100) * 0.60;
    }
    else if (calls <= 200)
    {
        bill = 200 + (50 * 0.60) + (calls - 150) * 0.50;
    }
    else
    {
        bill = 200 + (50 * 0.60) + (50 * 0.50) + (calls - 200) * 0.40;
    }

    cout << "Your telephone bill is: Rs. " << bill << endl;

    return 0;
}
```

```
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
Enter the number of calls made: 100
Your telephone bill is: Rs. 200
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
Enter the number of calls made: 300
Your telephone bill is: Rs. 295
PS C:\C++ PROGRAMS>
```

Question 4: Write a program to check the strength of a password entered by the user. The strength of the password is determined based on the following criteria:

- **Minimum length of 8 characters.**
- **Contains at least one uppercase letter, one lowercase letter, one digit, and one special character.**



C: > C++ PROGRAMS > tut1.cpp > ...

```cpp
int main()
    for (int i = 0; i < password.length(); i++)
            hasDigit = true;
        else
            hasSpecial = true;
    }

    // Check all conditions
    if (password.length() >= 8 && hasUpper && hasLower && hasDigit && hasSpecial)
    {
        cout << "Password is strong." << endl;
    }
    else
    {
        cout << "Password is weak. Make sure it:" << endl;
        if (password.length() < 8)
            cout << "- Has at least 8 characters" << endl;
        if (!hasUpper)
            cout << "- Has at least one uppercase letter" << endl;
        if (!hasLower)
            cout << "- Has at least one lowercase letter" << endl;
        if (!hasDigit)
            cout << "- Has at least one digit" << endl;
        if (!hasSpecial)
            cout << "- Has at least one special character" << endl;
    }

    return 0;
}
```

```
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
Enter your password: hello
Password is weak. Make sure it:
- Has at least 8 characters
- Has at least one uppercase letter
- Has at least one digit
- Has at least one special character
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
Enter your password: hello123$#
Password is weak. Make sure it:
- Has at least one uppercase letter
PS C:\C++ PROGRAMS>
```

Question 5: Write a program to encrypt and decrypt a text file using a simple encryption algorithm. The encryption algorithm involves shifting each character by a fixed number of positions in the ASCII character set.

```cpp
#include <iostream>
#include <fstream>
using namespace std;

// Function to encrypt a file
void encryptFile(string inputFile, string outputFile, int key) {
    ifstream inFile(inputFile);
    ofstream outFile(outputFile);
    char ch;

    if (!inFile || !outFile) {
        cout << "Error opening file!" << endl;
        return;
    }

    while (inFile.get(ch)) {
        ch = ch + key; // Shift character by key positions
        outFile.put(ch);
    }

    cout << "File encrypted successfully!" << endl;

    inFile.close();
    outFile.close();
}

// Function to decrypt a file
void decryptFile(string inputFile, string outputFile, int key) {
    ifstream inFile(inputFile);
    ofstream outFile(outputFile);
    char ch;

    if (!inFile || !outFile) {
        cout << "Error opening file!" << endl;
        return;
    }

    while (inFile.get(ch)) {
        ch = ch - key; // Shift back to original character
        outFile.put(ch);
    }

    cout << "File decrypted successfully!" << endl;

    inFile.close();
    outFile.close();
}

int main() {
    string original = "original.txt";
    string encrypted = "encrypted.txt";
    string decrypted = "decrypted.txt";
    int key;

    cout << "Enter the encryption key (number of positions to shift): ";
    cin >> key;

    encryptFile(original, encrypted, key);   // Encrypt original.txt to encrypted.txt
    decryptFile(encrypted, decrypted, key);  // Decrypt encrypted.txt to
decrypted.txt
    return 0;
}
```

Question 6: Write a C++ program to create a simple menu-driven calculator that performs basic arithmetic operations (addition, subtraction, multiplication, division).

TERMINAL   CHAT

```
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRA
MS\" ; if ($?) { g++ tut1.cpp -o tut1
 } ; if ($?) { .\tut1 }

=== Simple Calculator Menu ===
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice (1-5): 1
Enter first number: 2
Enter second number: 5
Result: 7

=== Simple Calculator Menu ===
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice (1-5): 5
Exiting calculator. Goodbye!
PS C:\C++ PROGRAMS>
```

```cpp
#include <iostream>
using namespace std;

int main() {
    int choice;
    float num1, num2, result;

    do {
        // Display menu
        cout << "\n=== Simple Calculator Menu ===\n";
        cout << "1. Addition\n";
        cout << "2. Subtraction\n";
        cout << "3. Multiplication\n";
        cout << "4. Division\n";
        cout << "5. Exit\n";
        cout << "Enter your choice (1-5): ";
        cin >> choice;

        // Perform operation
        if (choice >= 1 && choice <= 4) {
            cout << "Enter first number: ";
            cin >> num1;
            cout << "Enter second number: ";
            cin >> num2;
        }

        switch (choice) {
            case 1:
                result = num1 + num2;
                cout << "Result: " << result << endl;
                break;
            case 2:
                result = num1 - num2;
                cout << "Result: " << result << endl;
                break;
            case 3:
                result = num1 * num2;
                cout << "Result: " << result << endl;
                break;
            case 4:
                if (num2 != 0)
                    result = num1 / num2;
                else {
                    cout << "Error: Division by zero!" << endl;
                    break;
                }
                cout << "Result: " << result << endl;
                break;
            case 5:
                cout << "Exiting calculator. Goodbye!" << endl;
                break;
            default:
                cout << "Invalid choice! Please enter 1 to 5." <<
endl;    }
    } while (choice != 5);

    return 0;
}
```

Question 7: Write a C++ program to generate the Fibonacci series up to a given number of terms.

C: > C++ PROGRAMS > tut1.cpp > main()

```cpp
#include <iostream>
using namespace std;

int main()
{
    int n, first = 0, second = 1, next;

    cout << "Enter the number of terms: ";
    cin >> n;

    cout << "Fibonacci Series: ";

    for (int i = 1; i <= n; i++)
    {
        cout << first << " ";
        next = first + second;
        first = second;
        second = next;
    }

    return 0;
}
```

```
PS C:\Users\hp> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
Enter the number of terms: 7
Fibonacci Series: 0 1 1 2 3 5 8
PS C:\C++ PROGRAMS>
```

Question 8: Write a C++ program to implement a number guessing game where the user tries to guess a randomly generated number.

```
PS C:\Users\hp> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
Welcome to the Number Guessing Game!
I have selected a number between 1 and 100. Try to guess it!
Enter your guess (Attempt 1 of 10): 3
Too low! Try again.
Enter your guess (Attempt 2 of 10): 44
Too high! Try again.
Enter your guess (Attempt 3 of 10): 32
Too high! Try again.
Enter your guess (Attempt 4 of 10): 22
Too low! Try again.
Enter your guess (Attempt 5 of 10): 27
Too high! Try again.
Enter your guess (Attempt 6 of 10): 26
Congratulations! You guessed the correct number: 26
It took you 6 attempts.
PS C:\C++ PROGRAMS>
```

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>

using namespace std;

int main() {
    // Initialize random seed
    srand(time(0));

    // Generate a random number between 1 and 100
    int numberToGuess = rand() % 100 + 1;
    int userGuess = 0;
    int attempts = 0;
    const int maxAttempts = 10;  // Maximum attempts allowed

    cout << "Welcome to the Number Guessing Game!" << endl;
    cout << "I have selected a number between 1 and 100. Try to guess it!" << endl;

    // Loop until the user guesses the correct number or exceeds max attempts
    while (userGuess != numberToGuess && attempts < maxAttempts) {
        cout << "Enter your guess (Attempt " << attempts + 1 << " of " << maxAttempts << "): ";
        cin >> userGuess;
        attempts++;

        if (userGuess < numberToGuess) {
            cout << "Too low! Try again." << endl;
        } else if (userGuess > numberToGuess) {
            cout << "Too high! Try again." << endl;
        } else {
            cout << "Congratulations! You guessed the correct number: " << numberToGuess <<
endl;       cout << "It took you " << attempts << " attempts." << endl;
            break; // Exit the loop once the correct number is guessed
        }

        // Check if the maximum number of attempts has been reached
        if (attempts == maxAttempts) {
            cout << "Sorry! You've reached the maximum number of attempts." << endl;
            cout << "The correct number was " << numberToGuess << "." << endl;
        }
    }

    return 0;
}
```

Q9) Write a C++ program to implement a simple rock, paper, scissors game between the user and the computer.

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>

using namespace std;

int main() {
    // Initialize random seed
    srand(time(0));

    // Declare the choices
    string choices[] = {"Rock", "Paper", "Scissors"};

    // Variable to store user's choice and computer's choice
    string userChoice, computerChoice;

    // Get user's choice
    cout << "Welcome to Rock, Paper, Scissors!" << endl;
    cout << "Enter your choice (Rock, Paper, or Scissors): ";
    cin >> userChoice;

    // Convert user's input to lowercase to make it case-insensitive
    for (char &c : userChoice) {
        c = tolower(c);
    }

    // Check if the user entered a valid choice
    if (userChoice != "rock" && userChoice != "paper" && userChoice != "scissors") {
        cout << "Invalid choice! Please enter Rock, Paper, or Scissors." << endl;
        return 0;
    }

    // Get computer's choice (randomly)
    int computerChoiceIndex = rand() % 3;
    computerChoice = choices[computerChoiceIndex];

    // Display both choices
    cout << "Your choice: " << userChoice << endl;
    cout << "Computer's choice: " << computerChoice << endl;

    // Determine the winner
    if (userChoice == "rock" && computerChoice == "Scissors") {
        cout << "You win! Rock beats Scissors." << endl;
    } else if (userChoice == "paper" && computerChoice == "Rock") {
        cout << "You win! Paper beats Rock." << endl;
    } else if (userChoice == "scissors" && computerChoice == "Paper") {
        cout << "You win! Scissors beats Paper." << endl;
    } else if (userChoice == computerChoice) {
        cout << "It's a tie! Both chose " << computerChoice << "." << endl;
    } else {
        cout << "You lose! " << computerChoice << " beats " << userChoice << "." <<
endl}

    return 0;
}
```

```
TERMINAL    CHAT

PS C:\Users\hp> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if
 ($?) { .\tut1 }
Welcome to Rock, Paper, Scissors!
Enter your choice (Rock, Paper, or Scissors): Rock
Your choice: rock
Computer's choice: Scissors
You win! Rock beats Scissors.
PS C:\C++ PROGRAMS>
```

Question 10: Write a C++ program to display the name of the day of the week based on the day number entered by the user.

```cpp
#include <iostream>
using namespace std;

int main() {
    int dayNumber;

    // Ask the user to enter a day number (1-7)
    cout << "Enter the day number (1 for Monday, 7 for Sunday): ";
    cin >> dayNumber;

    // Display the name of the day based on the input
    switch (dayNumber) {
        case 1:
            cout << "Monday" << endl;
            break;
        case 2:
            cout << "Tuesday" << endl;
            break;
        case 3:
            cout << "Wednesday" << endl;
            break;
        case 4:
            cout << "Thursday" << endl;
            break;
        case 5:
            cout << "Friday" << endl;
            break;
        case 6:
            cout << "Saturday" << endl;
            break;
        case 7:
            cout << "Sunday" << endl;
            break;
        default:
            cout << "Invalid input! Please enter a number between 1 and 7." <<
endl}

    return 0;
}
```

```
PS C:\Users\hp> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if
 ($?) { .\tut1 }
Enter the day number (1 for Monday, 7 for Sunday): 1
Monday
PS C:\C++ PROGRAMS> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 }
; if ($?) { .\tut1 }
Enter the day number (1 for Monday, 7 for Sunday): 2
Tuesday
PS C:\C++ PROGRAMS>
```

Question 11: Write a C++ program to print the following patterns and shapes:

C: > C++ PROGRAMS > tut1.cpp > main()

```cpp
#include <iostream>
using namespace std;

int main()
{
    // Define the size of the square
    int squareSize = 5;

    // Pattern 4: Hollow Square
    cout << "Pattern 4: Hollow Square" << endl;
    for (int i = 1; i <= squareSize; i++)
    {
        for (int j = 1; j <= squareSize; j++)
        {
            if (i == 1 || i == squareSize || j == 1 || j == squareSize)
            {
                cout << "*"; // Print stars at the boundary
            }
            else
            {
                cout << " "; // Print spaces inside the square
            }
        }
        cout << endl;
    }

    cout << endl;
    return 0;
}
```

```
PS C:\Users\hp> cd "c:\C++ PROGRAMS\" ; if ($?) { g++
 tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
Pattern 4: Hollow Square
*****
*   *
*   *
*   *
*****

PS C:\C++ PROGRAMS>
```

C: > C++ PROGRAMS > tut1.cpp > main()

```cpp
int main()
    for (int i = 0; i <= mid; i++)
        for (int j = 0; j < mid - i; j++)
        }
        // Print stars
        for (int k = 0; k < (2 * i + 1); k++)
        {
            cout << "*";
        }
        cout << endl;
    }

    // Lower part of the diamond
    for (int i = mid - 1; i >= 0; i--)
    {
        // Print spaces
        for (int j = 0; j < mid - i; j++)
        {
            cout << " ";
        }
        // Print stars
        for (int k = 0; k < (2 * i + 1); k++)
        {
            cout << "*";
        }
        cout << endl;
    }

    return 0;
}
```

```
PS C:\Users\hp> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut
1.cpp -o tut1 } ; if ($?) { .\tut1 }
Enter the number of rows for the diamond (odd number): 7
   *
  ***
 *****
*******
 *****
  ***
   *
PS C:\C++ PROGRAMS>
```

C: > C++ PROGRAMS > tut1.cpp > main()

```cpp
#include <iostream>
using namespace std;

int main()
{
    int rows;

    // Ask the user to enter the number of rows for the right-angled triangle
    cout << "Enter the number of rows for the right-angled triangle: ";
    cin >> rows;

    // Loop through each row
    for (int i = 1; i <= rows; i++)
    {
        // Print stars in each row
        for (int j = 1; j <= i; j++)
        {
            cout << "*";
        }
        // Move to the next line after printing stars in a row
        cout << endl;
    }

    return 0;
}
```

```
PS C:\Users\hp> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut
1.cpp -o tut1 } ; if ($?) { .\tut1 }
Enter the number of rows for the right-angled triangle: 5
*
**
***
****
*****
PS C:\C++ PROGRAMS>
```

C: > C++ PROGRAMS > tut1.cpp > main()

```cpp
int main()
    int rows = 5; // Number of rows for the pyramid

    // Loop through each row
    for (int i = 1; i <= rows; i++)
    {
        // Print spaces for pyramid alignment
        for (int j = 1; j <= rows - i; j++)
        {
            cout << " ";
        }

        // Print the decreasing numbers
        for (int j = i; j > 0; j--)
        {
            cout << j;
        }

        // Print the increasing numbers
        for (int j = 2; j <= i; j++)
        {
            cout << j;
        }

        // Move to the next line after printing a row
        cout << endl;
    }

    return 0;
}
```

```
PS C:\Users\hp> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut
1.cpp -o tut1 } ; if ($?) { .\tut1 }
    1
   212
  32123
 4321234
543212345
PS C:\C++ PROGRAMS>
```

```
PS C:\Users\hp> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut
1.cpp -o tut1 } ; if ($?) { .\tut1 }
Enter the number of rows for the butterfly: 7
*            *
**          **
***        ***
****      ****
*****    *****
******  ******
**************
**************
******  ******
*****    *****
****      ****
***        ***
**          **
*            *
PS C:\C++ PROGRAMS>
```

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;

    // Ask the user for the size of the butterfly (number of
rows)cout << "Enter the number of rows for the butterfly: ";
    cin >> n;

    // Top half of the butterfly
    for (int i = 1; i <= n; i++) {
        // Print stars in the first part of the row
        for (int j = 1; j <= i; j++) {
            cout << "*";
        }

        // Print spaces between the two parts of the butterfly
        for (int j = 1; j <= 2 * (n - i); j++) {
            cout << " ";
        }

        // Print stars in the second part of the row
        for (int j = 1; j <= i; j++) {
            cout << "*";
        }

        // Move to the next line after each row
        cout << endl;
    }

    // Bottom half of the butterfly
    for (int i = n; i >= 1; i--) {
        // Print stars in the first part of the row
        for (int j = 1; j <= i; j++) {
            cout << "*";
        }

        // Print spaces between the two parts of the butterfly
        for (int j = 1; j <= 2 * (n - i); j++) {
            cout << " ";
        }

        // Print stars in the second part of the row
        for (int j = 1; j <= i; j++) {
            cout << "*";
        }

        // Move to the next line after each row
        cout << endl;
    }

    return 0;
}
```